

Maria Protas

Lead Developer / Senior Full-Stack Engineer (Python, Go) / ML

Telegram @vkusniypirogok

GitHub

Fulmo

LinkedIn

Maria Protas

Email

i@mprotas.ru

Location Nizhny Novgorod, Russia

Open to Remote / Relocation



At a glance

 Experience	 Companies	 Team led	 Level	 Availability
7+ years	Intel · YADRO · Transneft	11 engineers	Senior / Lead	Remote · UTC+3

English C1 · overlaps 2–4 hours with CET/EET working hours · open to relocation

Profile

Engineer with 7+ years of commercial experience, currently leading a team of 11 engineers. I still write code every day — leadership was added to the engineering work, not substituted for it.

Backend is my core specialisation: Python (advanced) and Go (working knowledge, production services since 2020). Microservices, gRPC contracts, asynchronous messaging over Kafka and RabbitMQ, PostgreSQL tuning under load. I design whole systems: service boundaries, data model, fault tolerance, deployment, monitoring.

Full-stack in production: React and TypeScript SPAs on top of my own APIs, Next.js where SSR pays off, Vue 3 where history left it.

Engineering culture: tests as part of the definition of done (coverage 20% → 75%+ on the current platform), mandatory code review, static type checking in CI.

SDET side: I build test infrastructure myself and treat it as code — fixtures and data factories, integration tests on testcontainers, stubs for external APIs, OpenAPI contract checks, Playwright UI automation, parallel runs and reports in CI. Quality is not a role standing next to development but a property of the pipeline: until the suite runs on every merge, “done” has not happened.

ML and deep learning — a second track, not a footnote: at Intel and YADRO I trained and fine-tuned models (TensorFlow / Keras, PyTorch, scikit-learn), ran hyperparameter search and solved regression and classification problems end to end — from dataset preparation to inference in production.

Open to: **Lead Developer · Team Lead · Senior Backend (Python / Go) · Senior Full-Stack · SDET / Test Automation · ML Engineer**

Tech stack

Backend

Python (advanced)	●●●●●	FastAPI 0.115+ · Django 4.2–6.1 / DRF 3.15+ · asyncio · Celery · SQLAlchemy async · Pydantic 2
HTTP clients	●●●●●	httpx 0.28 (async, timeouts, retries, pools) · requests · aiohttp
Authentication	●●●●●	Keycloak 26 (OIDC / SSO) · OAuth2 · JWT · roles and permissions
Go (working)	●●●●●	Gin · Echo · goroutines · channels · context · sync · gRPC · static binaries
gRPC / protobuf	●●●●●	service-to-service contracts · streaming · client generation
Microservices	●●●●●	service boundaries · API gateway · idempotency · graceful degradation
Message queues	●●●●●	Kafka (topics, partitions, consumer groups) · RabbitMQ (exchanges, DLQ, retries)
REST API	●●●●●	OpenAPI · Swagger · WebSocket · versioning
Node.js	●●●●●	Express · Nest.js – services and BFF layers for the frontend
Rust	●●○○○	side projects and CLI tools (~2 years): ownership, tokio, serde, cargo

Frontend

React	●●●●●	React 18+ · hooks · context · Redux/Zustand · React Query
TypeScript	●●●●●	strict · generics · typed API contracts · ES2022+
Next.js	●●●●●	SSR/SSG · App Router · load-time optimisation
Vue.js / Nuxt	●●●●●	Vue 3 · Composition API · Pinia · Nuxt 3
HTML5 / CSS3	●●●●●	Tailwind CSS · responsive layout · accessibility
React Native	●●○○○	Expo · mobile screens over a shared API (side projects)

Databases

PostgreSQL	●●●●●	query plans · indexes · partitioning · transactions and locking
MongoDB	●●●●●	motor (async) · aggregation pipelines · schema and index design
Redis	●●●●●	caching · pub/sub · task queues · distributed locks

System design

Architecture	●●●●●	monolith decomposition · sync vs async trade-offs · data modelling
Reliability	●●●●●	timeouts · retries with backoff · circuit breakers · idempotency
Scalability	●●●○○	caching · connection pooling · profiling bottlenecks

QA / SDET and code quality

pytest	●●●●●	unit · parametrisation · fixtures · mocks · coverage (pytest-cov)
pytest ecosystem	●●●●●	pytest-asyncio · pytest-mock · pytest-xdist (parallel runs) · Allure reports
Integration	●●●●●	testcontainers · test databases · WireMock and external-API stubs
API contracts	●●●●●	OpenAPI contract testing · schemathesis · Postman / Newman
UI automation	●●●●●	Playwright · Jest / Vitest
Test data	●●●○○	Faker · factory-boy · hypothesis (property-based)
Load testing	●●●○○	Locust · k6 – load profiles, finding bottlenecks
Process	●●●●●	code review · CI quality gate · Ruff · MyPy · basedpyright (strict) · pre-commit

DevOps and infrastructure

Docker / K8s	●●●○○	multi-stage builds · Docker Compose · Helm · Kubernetes
CI/CD	●●●●●	GitLab CI · GitHub Actions · tests and linters on every PR
Linux / Bash	●●●●●	automation · scripting · server administration
Observability	●●●●●	Grafana · Prometheus · ELK · structured logging

SRE

Kubernetes	●●●●○	1.30+ · Helm · rollouts, resource limits, health probes
CI/CD	●●●●○	GitLab CI · GitHub Actions · quality gate, tests on every merge
Monitoring	●●●●○	Grafana · Prometheus – dashboards, alerts, service metrics
SRE practices	●●●●○	SLA/SL0 · scheduled on-call · incident reviews and postmortems

AI / ML

LLM / RAG	●●●●○	OpenAI · Anthropic (Claude) API · RAG · function calling · LangChain
Deep learning	●●●●○	PyTorch · TensorFlow · Keras – CNN/RNN, training and fine-tuning
Classical ML	●●●●○	scikit-learn – regression, classification, cross-validation, metrics
Model training	●●●●○	dataset preparation · hyperparameter search · regularisation · production inference
MLOps	●●●○●	model versioning · scheduled retraining · weight rollout · post-release quality control

Git Nginx Jira Confluence Agile Scrum System Design Code Review

Experience

Transneft Technologies — Tech Lead / Lead Full-Stack Developer

Jun 2025 — Present · *IT arm of Transneft, the national oil & gas pipeline operator*

- 🏗️ **Designed the platform architecture:** decomposed the monolith into services, defined boundaries and contracts, chose synchronous (gRPC) or asynchronous (queues) interaction per case
- 🐍 **Built the Python 3.12–3.14 backend:** FastAPI 0.115+ and asyncio, SQLAlchemy async, Celery — the core business services
- 🐼 **Built Go 1.22–1.24 services** for integrations and background processing on Gin, where latency and concurrency matter — goroutines, channels, context cancellation; static binaries with no runtime on the target host
- 🚀 **Shipped the React SPA in TypeScript** on top of those APIs — forms, server-paginated tables, role-based access control
- 🔄 **Introduced asynchronous messaging** over Kafka and RabbitMQ: idempotent handlers, dead-letter queues, retry policies
- ✅ **Made quality a process:** pytest plus integration tests on testcontainers, a CI quality gate and mandatory review — raised coverage from **20% to 75%+**
- 🔧 **Built the platform's test infrastructure (SDET):** a layer of fixtures and data factories (Faker, factory-boy), dependencies spun up in testcontainers, stubs for external APIs, OpenAPI contract checks (schemathesis), parallel runs (pytest-xdist) and Allure reports in GitLab CI; Locust load profiles for business-critical endpoints before rollout
- 📈 **Cut response time of business-critical APIs by 40%** by rewriting query plans and adding covering indexes in PostgreSQL
- 👥 **Led a team of 11 engineers:** task decomposition, code review, mentoring, 1-on-1s, hiring
- 🤖 **Integrated LLMs** (OpenAI / Anthropic Claude API, RAG, function calling) into internal services

- 🧠 **Owned ML and MLOps on platform data:** forecasting and classification models (PyTorch 2.x, scikit-learn 1.5+), hyperparameter search, model versioning and serving in production, quality monitoring after each rollout
- 🛡️ **Ran the SRE side:** SLA/SLO for critical services, Grafana/Prometheus alerting, incident reviews and postmortems, scheduled on-call; Kubernetes 1.30+, GitLab CI
- 🔗 **Built integrations and access control:** httpx async clients (timeouts, retries, connection pools), single sign-on via Keycloak 26 (OIDC/SSO, JWT, roles)

Python FastAPI httpx Keycloak Go gRPC Kafka RabbitMQ React TypeScript PostgreSQL
Redis Docker K8s GitLab CI LLM PyTorch scikit-learn

🟡 YADRO — Senior Full-Stack Developer

Feb 2023 — Jun 2025 · Hardware R&D company — servers, storage systems and telecom equipment

- 🌐 **Built a web platform** for hardware and software performance monitoring: Django 4.2–5.1 / DRF 3.14+ with Vue 3 and TypeScript 5
- ⚡ **Delivered REST and WebSocket APIs** for real-time metrics; collected and aggregated device telemetry over httpx (async, timeouts, retries)
- 🧠 **Trained and fine-tuned models on telemetry** (PyTorch 2.0–2.5, TensorFlow / Keras 2.12+, scikit-learn 1.3+): hyperparameter search, regression and classification tasks, evaluation on held-out data
- 🛠️ **Operated the platform:** builds and releases through GitLab CI, dashboards and alerts in Grafana / Prometheus, on-call duty and incident analysis
- 🗑️ **Rolled out single sign-on** and fine-grained permissions via Keycloak (OIDC/SSO)
- 🐹 **Wrote metric-collection tools in Go 1.20–1.23** where a single static binary with no runtime on the target machine was required
- 🧩 **Maintained a Vue 3 component library** (Composition API, Pinia) and OpenAPI/Swagger contracts
- ✅ **Automated regression across the stack:** pytest on the backend, Jest on the frontend, Playwright UI scenarios, all running on every merge in GitLab CI; code review, Scrum

Python Django 4–5 DRF httpx Keycloak Go Vue 3 TypeScript PostgreSQL MongoDB Docker
pytest Jest PyTorch TensorFlow scikit-learn

🟢 Intel Corporation — AI Systems Engineer / Full-Stack Developer

Feb 2018 — Feb 2022 · Semiconductor / AI & ML

- 🧠 **Trained and fine-tuned deep learning models** on TensorFlow 1.15–2.8 / Keras and PyTorch 1.x, plus classical algorithms on scikit-learn 0.20–1.0
- 🧩 **Owned the full model lifecycle:** dataset preparation and labelling, architecture selection, hyperparameter search, regularisation, regression and classification metrics, inference in production
- ⚡ **Built inference services and internal APIs on FastAPI** (from 2019): async prediction endpoints, Pydantic request validation, wired into the training pipelines
- 🕷️ **Built web tools** for software and hardware performance analysis: Python 3.6–3.9 (Django 2.0–4.0, FastAPI 0.4x–0.7x) with React 16 and D3.js

- 🇩🇪 **Backend work:** REST APIs, PostgreSQL 9.6–13 and MongoDB 4.x; dashboards over large result sets
- ♻️ **Automated training and retraining pipelines:** data preparation, scheduled retraining, weight rollout into services — MLOps before it was called that
- ✏️ **Automated testing of the analysis tooling** (pytest, unittest): regression suites over real datasets, checks that model metrics stayed stable between runs; monitoring in Grafana / Prometheus

Python Django 2-4 FastAPI React TypeScript D3.js PostgreSQL MongoDB Docker Grafana
TensorFlow Keras PyTorch scikit-learn Deep Learning

🚀 Side projects

Side projects running in production — there I own the whole system: data model, architecture, deployment, monitoring and recovery from failures.

📖 Interviews — offline interview-prep platform · **production**

Python · TypeScript · WebAssembly · Android · CI/CD

- **94 study modules and 258 exercises** with code execution right in the browser: **Pyodide** (Python compiled to WASM) and **sql.js** — no server, no internet connection
- **One source, three products:** Markdown and Jupyter notebooks build into a web app, a Telegram Mini App and an Android APK; build and APK release are automated in CI
- The systems problem was offline-first: the entire runtime, search index and syntax highlighting ship as static assets, progress is stored locally

Python TypeScript WebAssembly Pyodide Android GitHub Actions

💰 Financial dashboard over accounting data · **production**

Python · FastAPI · React · Plotly · pandas

- Web analytics on top of a 1C accounting database (the accounting standard used across Russia): reads **.1CD** files directly through onec_dtools — no COM layer, no load on the live database — with an in-memory cache of **~1.3M ledger entries**
- FastAPI backend and REST API: account turnover, counterparty correspondence, top counterparties, **ABC analysis** (80/15/5), year-over-year dynamics
- **Cash-gap forecasting** (scenario-based, 30/60/90 days); data health checks for duplicates and missing attributes
- Frontend: React SPA (Vite + TypeScript) with Plotly; Ruff, MyPy and basedpyright (strict), pre-commit, GitHub Actions CI

Python FastAPI React TypeScript Plotly pandas GitHub Actions

Very Match — order intake and analytics bot · **production**

Python · aiogram 3 · FastAPI · MongoDB · Docker

- End-to-end order intake bot: **4 FSM-driven scenarios**, a catalogue with modifiers pulled from an external API and synchronised every 6 hours
- **Async backend** on FastAPI and MongoDB (motor), repository and service layers, Pydantic 2; a customer-to-manager relay built on supergroup forum topics
- **APScheduler** jobs: escalations, payment reminders, delayed review requests, automated database backups
- **Web dashboard** (FastAPI + Chart.js): KPIs, conversion, customer segmentation; CSV/XLSX export
- Reliability: anti-spam middleware, health checks, automatic polling restart, log rotation
- **285 unit tests** · Ruff, MyPy, basedpyright · multi-stage Docker build (non-root) · CI

Python 3.14 **aiogram 3** **FastAPI** **MongoDB** **Pydantic 2** **APScheduler** **Docker** **pytest**

HProject — health bot with food recognition · **production**

Python · FastAPI · PostgreSQL · LLM

- Calorie and macro estimation from a photo of a meal: the pipeline runs **photo** → **LLM recognition** → **normalisation against a reference database** → **food diary**
- Designed around an unreliable external model: timeouts, retries, graceful degradation to manual entry when the LLM fails
- Food diary and statistics: aggregation by period, calorie targets

Python **FastAPI** **PostgreSQL** **LLM** **aiogram**

VPNTunnel — VPN router on sing-box · **self-hosted**

Python (stdlib) · asyncio · sing-box · launchd

- Parses subscriptions (VLESS/VMess/Trojan/Shadowsocks), async TCP-pings the server pool and picks the best one by availability → stability → throughput
- **Two-phase evaluation**: gate and stability checks for every server, real throughput measured only for the top 3; sticky selection and automatic config rollback on failure
- Systems side: sing-box config generation (TUN with typed DNS), autostart through launchd, Telegram notifications
- Pure stdlib, **69 tests**, ruff/mypy/basedpyright (strict) — zero errors

Python **asyncio** **sing-box** **launchd** **pytest**

zentist-rpa — RPA platform for external portals · **production**

Python · Playwright · PostgreSQL · Docker

- Automates work with portals that expose no public API: browser scenarios on Playwright
- Resilient to markup changes: several element-lookup strategies, retries, step-by-step journaling for failure analysis

- Job queue and execution reports

Python Playwright PostgreSQL Docker

Education

Degree	Year	Institution	Field
 M.Sc.	2021	Lobachevsky State University of Nizhny Novgorod (UNN)	Fundamental Informatics and Information Technology
 B.Sc.	2019	Lobachevsky State University of Nizhny Novgorod (UNN)	Applied Mathematics and Computer Science

Languages

 **Russian** — native ·  **English** — C1 (documentation, negotiations, business correspondence) ·  **Italian** — A2

Additional

Currently learning: Kubernetes (CKA), Apache Kafka, Rust (side projects and CLI tools, ~2 years), React Native

Work setup: Remote · hybrid in Nizhny Novgorod · open to relocation · business trips · project-based or part-time engagements

Open to roles: Lead Developer / Team Lead · Senior Backend (Python / Go) · Senior Full-Stack · SDET / Senior QA Automation · Senior Frontend (React / Vue)

Contact

Email	i@mprotas.ru
Telegram	@vkusniypirogok
GitHub	github.com/Fulmo
Location	Nizhny Novgorod, Russia — UTC+3, 2–4 hours of overlap with CET/EET

Updated: August 2026